

1. What is a convention (in the context of programming) and how does a convention differ from a rule or specification?

A common way for coders to write their code. It's different from a rule because it is not necessary for the code to work but it's something that encourages a more universal understanding of the program/readability.

2. How are comments indicated syntactically?

// for a single line

```
/*  
    for multiple lines  
    of comments  
*/
```

3. What role is played by whitespace characters (tabs, spaces, and newlines) in the source code?

Type script doesn't read whitespace so it is used for formatting and readability.

4. Is there a distinction made between expressions and statements? If so, explain the difference. How do side effects factor into this distinction, if at all?

Yes there is a difference. Expressions produce values and statements “tells the program what to do”. Expressions can have side effects where something can be changed while evaluating a value.

5. In TypeScript syntax, how are statements terminated? Is statement termination required or optional?

With a semi colon. Typescript can run without them, as it tries to figure out where semi-colons should be, but this can create unintended side effects with the code.

6. How are blocks of code (such as the body of a loop) indicated syntactically?

Curly brackets.

7. What is meant by the scope of a variable and why is it important for a programmer to understand?

Scope of a variable is where in the program the variable can be accessed. It is important to know because knowing it can stop programmers from accidentally changing variables in certain places or stop them from trying to access a variable outside of where they could access it.

8. What is a keyword? Are there keywords that are reserved and cannot be used as identifiers? Provide a link to the most complete and current list of reserved words in TypeScript that you can find.

A word with a “special meaning” in a coding language. Words such as “if” or “as” can't be used as identifiers. There are many which can't be used.

Link to list:

<https://finxact.io/docs/document-detail/document/typescript-language-keywords/>

9. Is it necessary to predeclare variables and, if so, how is it done?

They should be predeclared. In TypeScript you use var, let, and const. For example: let name = "paul";

10. What are the three statements that introduce variables and how do they differ? Which of the three is old and largely avoided by many modern JS programmers?

The words are var, let, and const.

var = The oldest and largely avoided. Old way of declaring variables. Handles scope differently.

let = Used when the variable may be changed.

const = Used when variable should not be changed.

11. Is variable initialization possible? If so, how is it done? Are there any special rules about it to keep in mind?

Yes, you give it a value when declaring. A special rule to keep in mind is const needs to be declared with a value.

12. Can you declare and/or initialize several variables with a single statement? If so, how?

Yes, you use commas.

13. What are the rules for writing valid identifiers for variables and functions? What conventions are applicable to identifiers?

Identifiers are case sensitive and can start with letters and the symbols "_" and "\$". Can not start with a number. A convention applicable to identifiers is called camelCase. An example of camelCase is findAverage.

14. What is strict mode? How do you enable it, and what is its scope when you do? Do you need to explicitly enable strict mode when your code will be run using Deno? Why or why not? Does strict mode add any words to the list of reserved words? If so, provide a URL where those additional words can be found (or list them). What other changes does strict mode entail, and why are they potentially useful? When, if ever, would you not want to use it and why?

It makes JavaScript use "stricter" rules. It is enabled with "use strict";.

If it is at the top of code it will apply to the code, if in a function it will only apply to the function.

You don't need to enable strict mode on Deno because it uses TS' strict mode by default. It adds extra reserved words which can be within this link:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode

It also prevents things like accidentally creating global variables. There really isn't much reason to avoid strict mode with new code, but old JS code might depend on things that strict mode doesn't allow.

15. What do the null, undefined, NaN, and Infinity keywords each represent and how might each be used?

null = no value (intentionally)

undefined = not assigned a value

NaN = "Not a Number", when something occurs that can't produce a value

Infinity = number beyond finite range.

16. Imagine you have a variable named myVar in a TypeScript program. How would you determine if the value stored in myVar is undefined?

You could use a function with ===. For example:

```
if(myVar === undefined) {  
    return("myVar is undefined");  
}
```

17. Imagine you have a variable named myVar in a TypeScript program. How would you determine if the value stored in myVar is null?

Same as 16 but replace undefined with null.

18. Imagine you have a variable named myVar in a TypeScript program. How would you determine if the value stored in myVar is NaN?

Number.isNaN(myVar), can't use === because NaN could never equal itself.

19. What arithmetic operators are available? Provide both their symbolic representation and their textual names.

+ addition, - subtraction, * multiplication, / division, % remainder, ** exponentiation, ++ increment, -- decrement

20. Are mathematical functions such as sine, round, minimum, natural logarithm, and absolute value available in the TypeScript programming environment? If so, how are they accessed?

Yes. They can be accessed by using Math objects. Such as Math.sin(value)

21. Are numerical constants such as pi, e, and the square root of 2 available in the TypeScript programming environment? If so, how are they accessed?

Also use Math to access. For example, Math.PI. They are capitalized.

22. Is a random number generator available? If so, how is it used and what are its limitations?

Math.random(). It give you a random decimal number from 0 up to but not including 1. The issue it is pseudorandom so it shouldn't be used for serious encryption.

23. What decision control structures are available and what are their respective syntaxes? What, if any, optional clauses may be used and how do they behave?

Main decision control structures are if, if...else, else if, and switch, using syntax such as if (condition) { code }, else if (condition) { code }, else { code }, and switch (expression) { case value: code; break; }. else and else if are optional for if statements, while default is optional for switch, and they allow the program to run different code when the original condition or cases don't match.

24. Is there a way to express a simple decision using an expression instead of a control structure? If so, what is its syntax?

A ternary operator. Syntax: condition ? expression_if_true : expression_if_false;

25. What Boolean operators are available? What are their symbolic representations and their meanings? Do any of them utilize lazy (or short-circuit) evaluation? If so, what does this mean?

&& And, || Or, ! Not. And and Or use lazy evaluation meaning once the program knows what the result is it doesn't check the rest of the expression.

26. What iteration control structures are available and what are their respective syntaxes? How does each one behave?

while repeats until condition is met.

```
while (condition) {  
    code  
}
```

A do...while loop is similar, except it always runs at least once because the condition is checked after the code runs.

```
do {  
    code  
} while (condition);
```

for loop is useful for when you know how you want the loop to count.

```
for (let i = 0; i < 10; i++) {  
    console.log(i);  
}
```

for...of can be used to go through the values of something like an array.

```
for (const item of items) {  
    console.log(item);  
}
```

27. Is it possible to exit a loop body and immediately proceed with the code following the loop? If so, how?

Yes. use break.

28. Is it possible to exit a loop body and proceed to that loop's next iteration (if there is one)? If so, how?

Use continue.

29. What is the purpose of a web resource such as Github or GitLab? Although we will be using the department's GitLab server for this course, when might you use Github instead?

They host Git repos and store changes/history to the repos. GitHub could be used when you do personal projects and things for employers.

30. How do you clone a repository from Github? How do you clone a repository from the department's local GitLab server?

For either one you can copy the repository URL and use `git clone <repourl>`.

31. Why do you suppose you first need to establish your own account on Github before you may access it? Why do you suppose you do not need to do so on the department's local GitLab server?

GitHub is its own service that isn't directly part of UNH, so you have to make your own account. The department GitLab is connected to UNH, so students can use their existing UNH account instead of creating a completely separate account.

32. What is the purpose of SSH keys and how do they relate to using Git with a service like Github or GitLab? Where can you find guidance on setting up SSH keys for authentication with the department's local GitLab server?

SSH keys are a way of proving who you are when connecting a computer to Github or GitLab. Guidance can be found here. <https://docs.gitlab.com/ee/user/ssh.html>

33. In the context of Git, what is a remote and when is one required? How do you list the existing remotes on the current repository? How do you add a new remote to the current repository?

A remote is connected to another copy of the repo. You need one when you want to push your local work somewhere else or pull work from another repository. You see them with `git remote -v`. You can add them with `git remote add <name> <URL>`

34. When working with a service such as Github or GitLab, what is pushing and how does it fit into the typical Git workflow?

Pushing means sending commits from your local repository to a remote repository like GitHub or GitLab. It is the last stage of a typical Git workflow.

35. What is meant by the term clean code? Why is writing clean code important to professional programmers? Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit.

Clean code is readable code that is easy for others to interpret and understand. It is important because professionals often work in big teams and the code needs to be clean so workflow can be easily transferred from one person to another. Some things from this unit that can help make code cleaner would be using good variable names, following conventions like camelCase, using whitespace to make my code readable, and not making code more complicated than it needs to be. Comments can also help explain parts of the code that aren't immediately obvious. Overall, clean code makes programs easier to understand, fix, and change later.